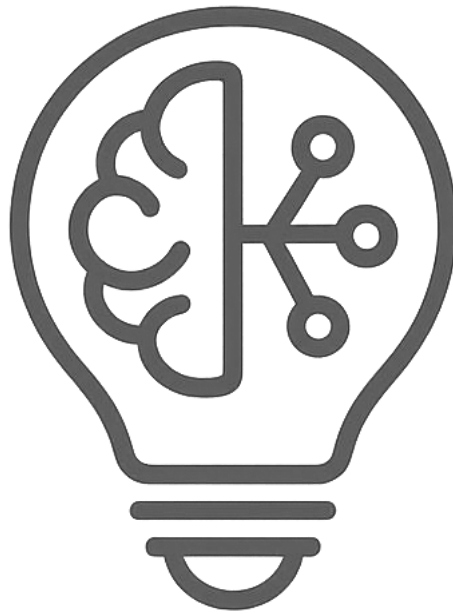




BrainBank

ISPW PROJECT - University of Rome Tor Vergata.

GIULIA MENCACCINI



BrainBank

Project Documentation

Table of contents

BrainBank– PROJECT DOCUMENTATION	0
1. Software Requirement Specification (SRS)	2
1.1 Introduction	2
1.1.1 Aim of the Document	2
1.1.2 Overview of the Defined System	2
1.1.3 Hardware and Software Requirements	2
1.1.4 Related Systems – Pros and Cons	3
1.2 User Stories	4
1.3 Functional Requirements	4
1.4 Use Cases	5
1.4.1 Overview Diagram	5
1.4.2 Internal Steps	6
2. Storyboards.....	7
3. Design.....	10
3.1 Class Diagram.....	10
3.1.1 VOPC analysis	10
3.1.2 Design-level Diagram per member.....	11
3.2 Design Patterns	13
3.3 Activity Diagram.....	14
3.4 Sequence Diagram.....	15
3.5 State Diagram	16
4. Testing	16
5. Code	18
6. Video.....	18

1. Software Requirement Specification (SRS)

1.1 Introduction

1.1.1 Aim of the Document

The purpose of this document is to present the fundamental and specific requirements of the “BrainBank” system, a project developed for the Software Engineering and Web Design course, Academic Year 2025–2026.

The document provides a comprehensive overview of the system’s objectives, core functionalities, and design choices in order to guide the development process.

Prepared by: Giulia Mencaccini

1.1.2 Overview of the Defined System

The system is a Java-based software platform structured according to the MVC/MVP architecture, developed using Maven, and featuring graphical user interfaces implemented through JavaFX and a command-line interface.

The system supports the efficient and modern management of booking lesson with a tutor, including the following functionalities:

- Management of students, tutors, and administrators.
- Booking and management of tutoring lessons based on subject and tutor availability.
- Visualization of tutor availability and lesson schedules.
- Assignment and management of activities between tutors and students.
- Monitoring of student progress and completion of assigned activities.
- Email notifications for booking confirmations, cancellations, and activity assignments.
- Viewing reports and statistical data to support administrative monitoring and decision-making processes.

There are three user types with different functionalities: **Students, Tutors, and Administrators.**

1.1.3 Hardware and Software Requirements

Software requirements:

- Java 17+
- Maven 3.8+
- MySQL Server 8,0+
- JavaFX 17+
- IDE: IntelliJ IDEA (recommended) or Eclipse
- Draw.io (for creating UML diagrams)

Hardware requirements:

- CPU: Dual-Core 2.0 GHz
- RAM: 4 GB (2 GB minimum)
- Disk Space: 1 GB
- Operating System: Windows, MacOS, Linux (compatible with java 17+)

1.1.4 Related Systems – Pros and Cons

BrainBank is designed as a university-distributed application, therefore it does not include a payment system.

BrainBank**PRO:**

- Free of charge. No payment system, accessible to all university students without economic barriers.
- Guided booking process with automatic Meet link generation.
- Real-time email notifications for booking confirmations, cancellations and activities assignment.

CONS:

- No tutor rating system.
- Desktop only, no mobile application available.
- Meet link is locally generated and not synchronized with external calendars such as Google Calendar.

Superprof

PRO	CONS
Offers a very large number of tutors and subjects, providing a wide range of choices for students.	The large number of available tutors can make the selection process difficult and overwhelming.
The platform has a clear and intuitive interface, making it easy to use.	The platform is based on a payment system, which may limit accessibility for some users.

leTueLezioni

PRO	CONS
Allows tutors to easily create profiles and offer lessons.	The system does not always clearly distinguish between tutors and students, which may cause confusion.
Covers a wide range of educational contexts, including schools and private training.	The search and filtering functionalities are limited, reducing usability.

1.2 User Stories

1. As a student, I want to book a lesson with a tutor by subject and availability, so that I can schedule a session that fits my needs.
2. As a tutor, I want to monitor the completion status of activities assigned to students, so that I can track their learning progress.
3. As an administrator, I want to view reports and statistics on platform usage, so that I can identify the most requested subjects and top tutors.

1.3 Functional Requirements

1. The system shall provide the student with a list of tutors associated with the selected subject.
2. The system shall display to the tutor the completion status of the activities assigned to students.
3. The system shall provide the administrator with the most requested subjects and top tutors.

Glossary

Slot= A predefined time interval (e.g., 60 minutes) created by a Tutor during which they are available to hold a lesson. Once booked, the slot is marked as unavailable.

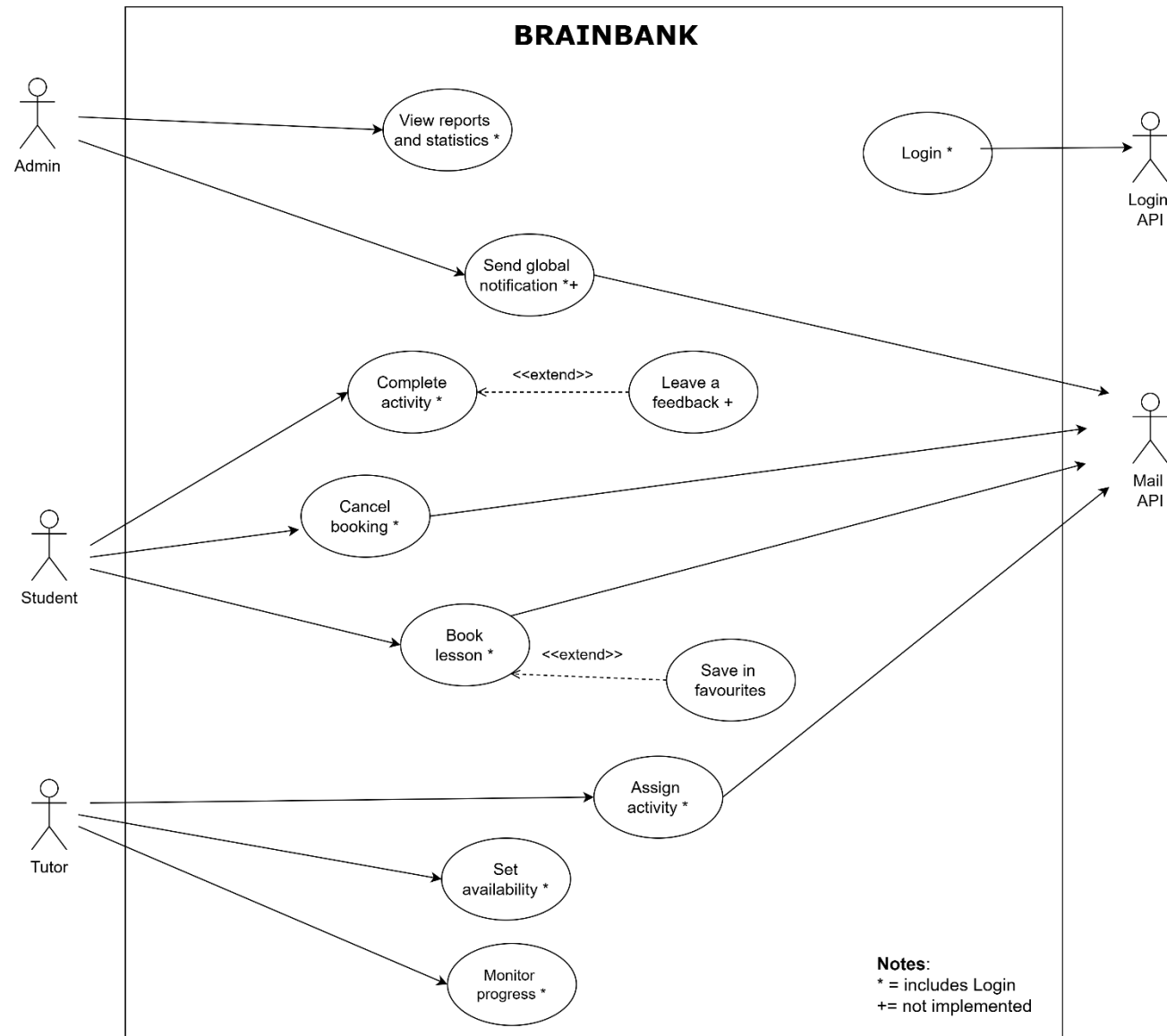
Booking= The formal association between a Student and a Tutor for a specific subject during an available Slot. Once confirmed, the slot is marked as unavailable and a Meeting link is automatically generated.

Activity= A specific task, exercise, or assignment created by a Tutor and assigned to a specific Student to support their learning progress.

Completion Status= The completion state of each activity assigned to a Student, indicating whether it has been completed or is still pending.

1.4 Use Cases

1.4.1 Overview Diagram



1.4.2 Internal Steps

Use case: Book Lesson

Main flow:

1. The student accesses their account (UC Login).
2. The student selects the Book Lesson function from the main menu.
3. The system displays the list of available subjects.
4. The student selects a subject of interest.
5. The system displays the list of available tutors for the selected subject, highlighting saved favourites.
6. The student selects a tutor.
7. The system displays the tutor's available slots.
8. The student selects a time slot for the desired lesson.
9. The system verifies the availability of the selected time slot and temporarily reserves it for 3 minutes.
10. The system displays a booking summary (date, time, tutor, and subject).
11. The student confirms the booking.
12. The system records the booking, generates a Meet link and sends a confirmation email both to the student and the tutor.

Extensions:

5a. No tutors available for the selected subject: The system notifies the student and returns them to the subject selection screen (step 3).

8a. The selected slot is no longer available: The system notifies the student and returns them to the slot selection screen (step 7).

11a. Booking timeout: The booking session expires before confirmation. The system displays a message indicating that the booking has been cancelled and terminates the use case.

2. Storyboards

Login

BrainBank

—

×

Benvenuto su

BrainBank
Connettiti al sapere!

Email

Password

☐ Mostra password

Login

Non hai ancora un account?
[Registrati qui](#)

Registrazione

BrainBank

—

×

◀◀

Indietro

Registrazione


Nome *
inserisci nome

Cognome *
inserisci cognome

Email *
inserisci email

* campi obbligatori

☒ **Studente**

☐ **Tutor**

Password*
inserisci password
La tua password deve includere

- almeno 8 caratteri
- una lettera maiuscola
- almeno un numero

Conferma Password *
ripeti password

☐ Mostra password

Registrami

Dashboard Studente

 BrainBank

 **Bentornata Emma!**

Studente

 **Log out**

Calendario Lezioni

GMT+01	DOM 9	LUN 10	MAR 11	MER 12	GIO 13	VEN 14
8 AM						
9 AM	FISICA 1 Bianchi 9 - 11AM				GEOMETRIA Rossi 9 - 11AM	
10 AM						
11 AM						
12 PM						
1 PM						
2 PM			ANALISI 2 Rossi 2 - 4PM			
3 PM						
4 PM						
5 PM						
6 PM						

 **Prenota Lezione**

 **Gestisci Lezioni**

 **To-do**

Prenota Lezione-Riepilogo Finale

 BrainBank

 **Indietro**

Prenota Lezione



Riepilogo prenotazione

Materia: Analisi 1

Tutor: Gabriele Rossi

Giorno: 20-11-2025

Slot selezionato: 9:00 - 11:00 AM

Prenota

 **08:58**

Dashboard Tutor

 BrainBank

 **Bentornato Gabriele!**

Tutor

 **Log out**

Calendario Lezioni

GMT+01	DOM 9	LUN 10	MAR 11	MER 12	GIO 13	VEN 14
8 AM						
9 AM						
10 AM						
11 AM						
12 PM						
1 PM						
2 PM						
3 PM						
4 PM						
5 PM						
6 PM						

 **Imposta disponibilità**

 **I miei slot**

 **Gestisci Studenti**

Gestisci Studenti

 BrainBank

 Home

Gestisci Studenti



Studente

Matteo Verdi 

Studente: Matteo Verdi  più info

Materie di interesse: Analisi 1, Fisica 1

Annota progressi
Capitolo 1 e 2 completati |
Aggiorna

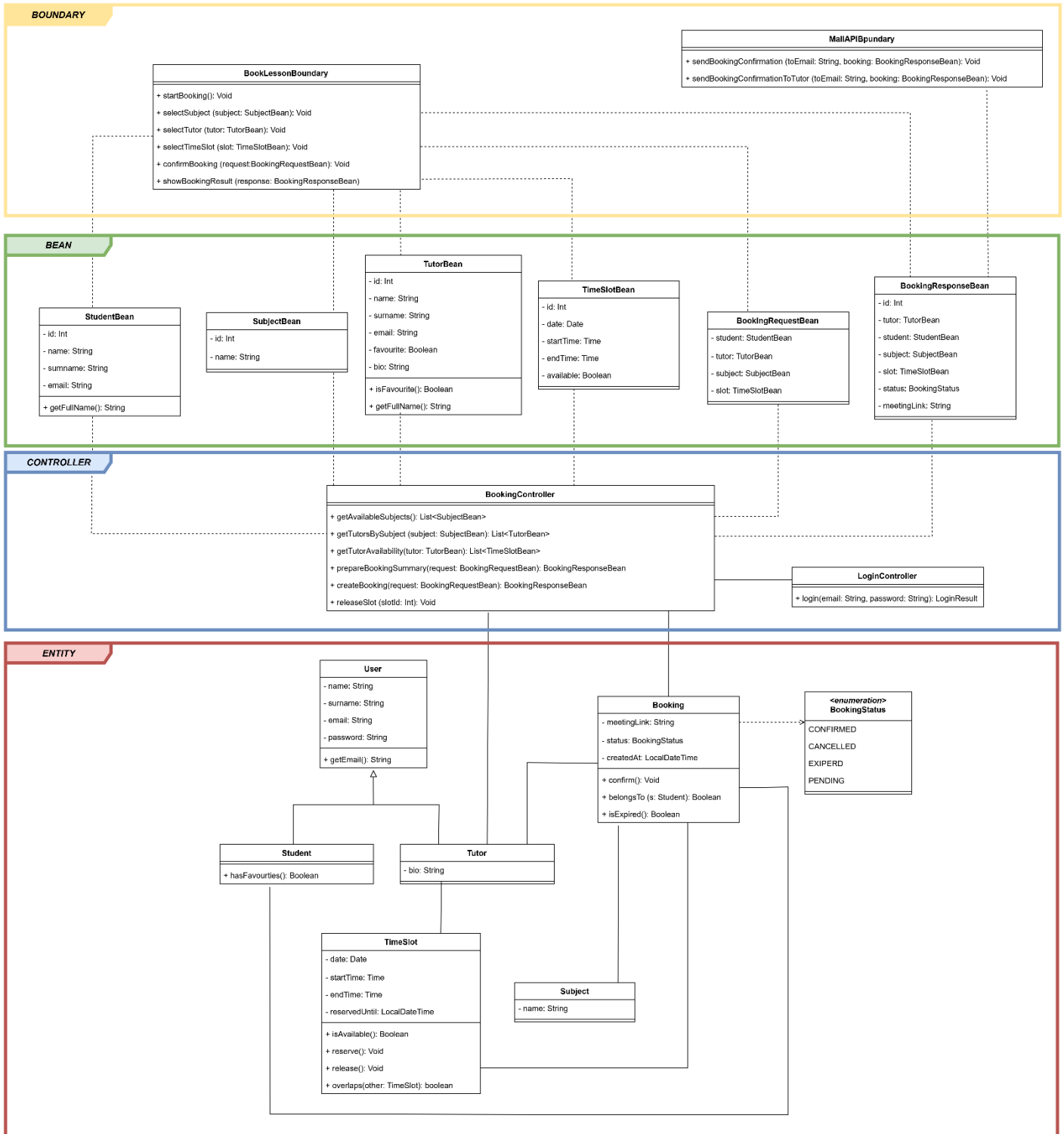
Assegna to do list
1. Esercizi 21-28
2. Ripasso Teoremi precedenti
+
Assegna

Ultimo aggiornamento 11 - 11- 2025

3. Design

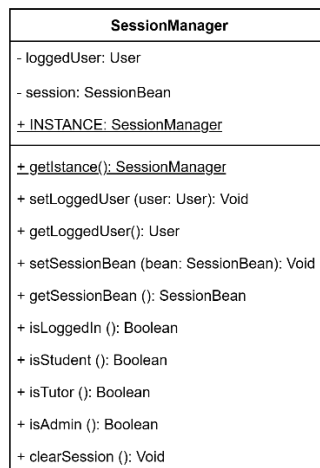
3.1 Class Diagram

3.1.1 VOPC analysis

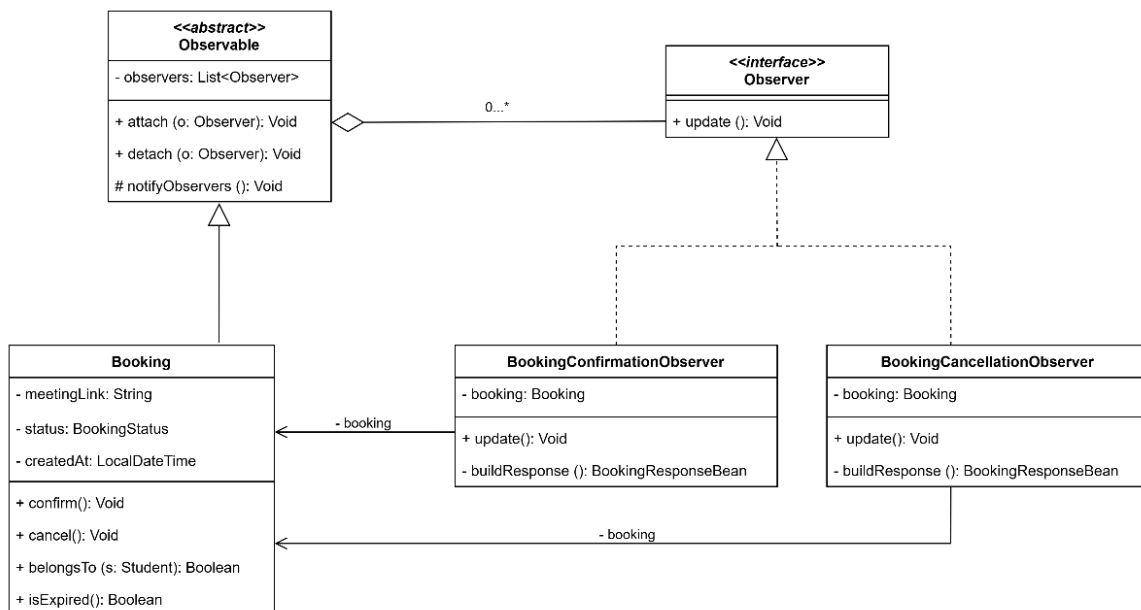


3.1.2 Design-level Diagram

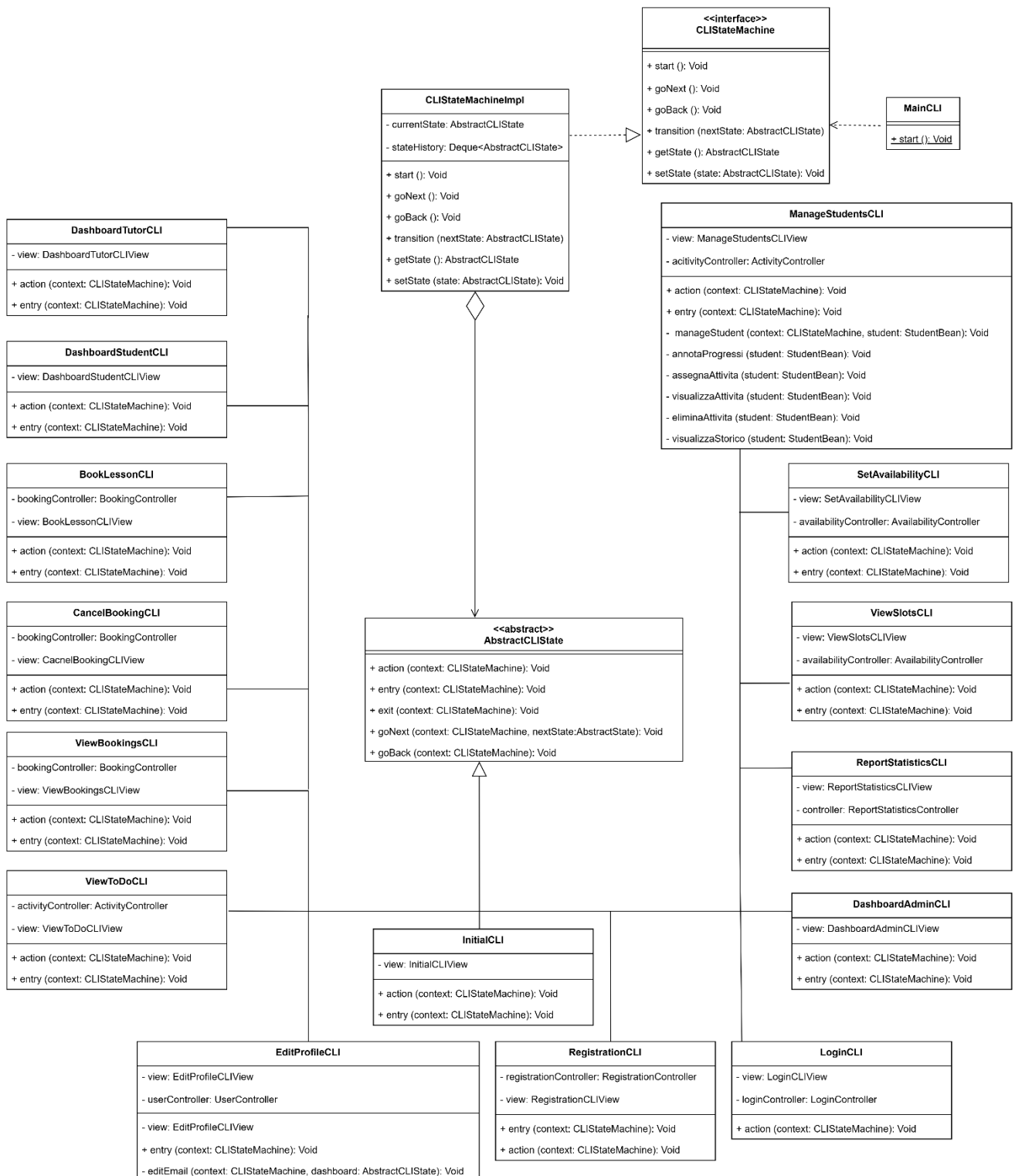
Singleton Pattern:



Observer Pattern:



State Pattern:



3.2 Design Patterns

1. SINGLETON

I adopted the Singleton pattern for SessionManager because the application needed a single, shared point to store session data (logged-in user and their role) accessible from any controller without passing it around explicitly. In fact, it guarantees that a class has exactly one instance across the entire application .

It was implemented using the Initialization on-demand Holder idiom. It consists in a private static inner class Holder that holds the unique instance, ensuring lazy initialization and thread safety without synchronized.

Furthermore, SessionManager exposes role checking methods that allow controllers to check the current user's role without accessing the User object directly and keeping authorization logic centralized.

2. OBSERVER

I used the Observer pattern to decouple booking state changes from notification handling. Neither Booking nor the BookingController know about NotificationService.

The observer, attached in BookingController before calling confirm() or cancel(), reacts to the state change and triggers the appropriate email notification to both student and tutor. Two concrete observers are defined: BookingConfirmationObserver and BookingCancellationObserver. Both read the updated state directly from the Booking instance inside update().

The design is easily extensible, new notification channels (e.g. SMS, push) or new state changes can be handled by simply adding a new observer without modifying existing classes.

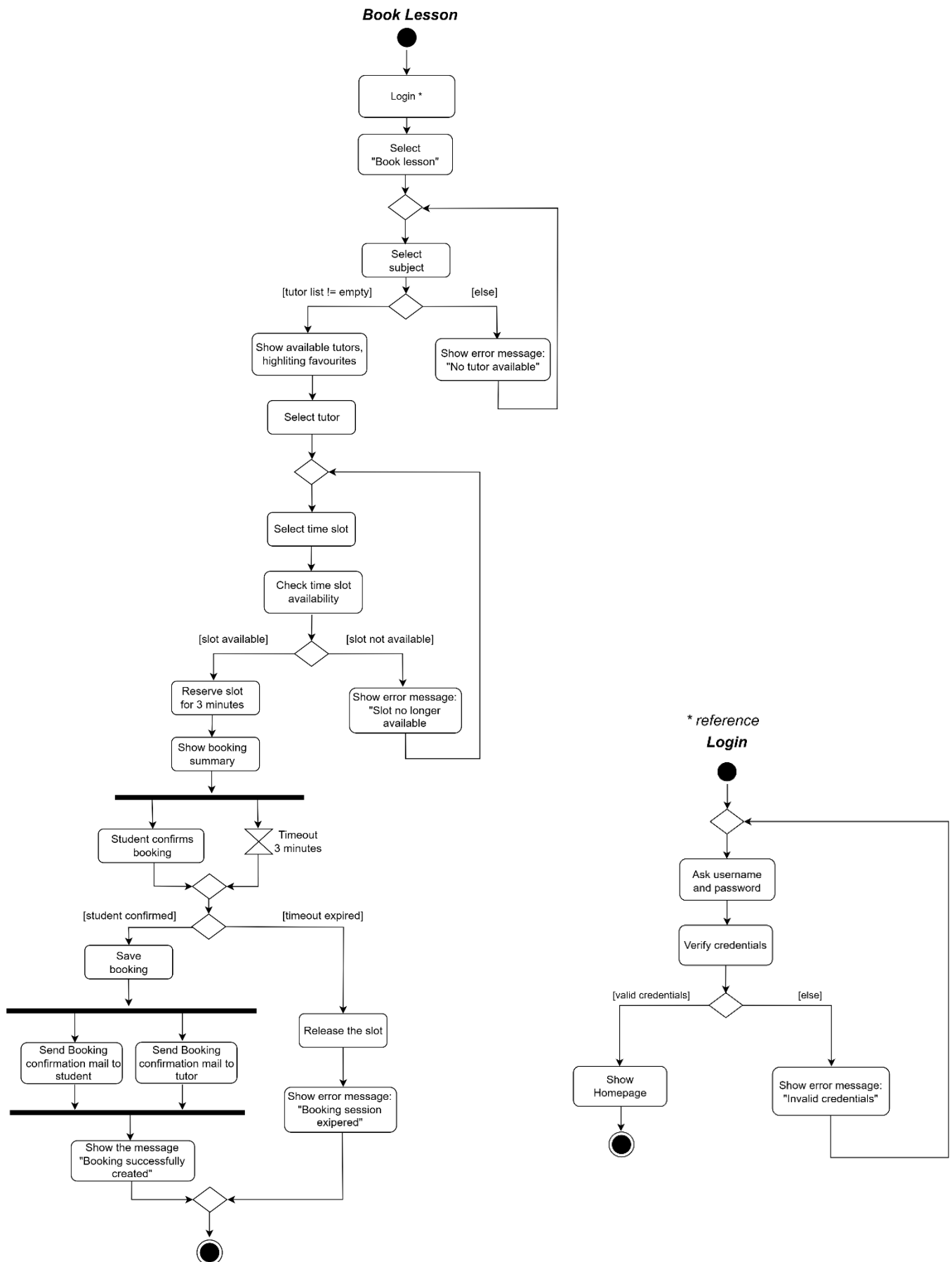
3. STATE

I chose the State pattern to replace a centralized navigation structure in which all CLI screens were managed through a single switch-case in MainCLI, with each screen returning an enum value to drive the next transition.

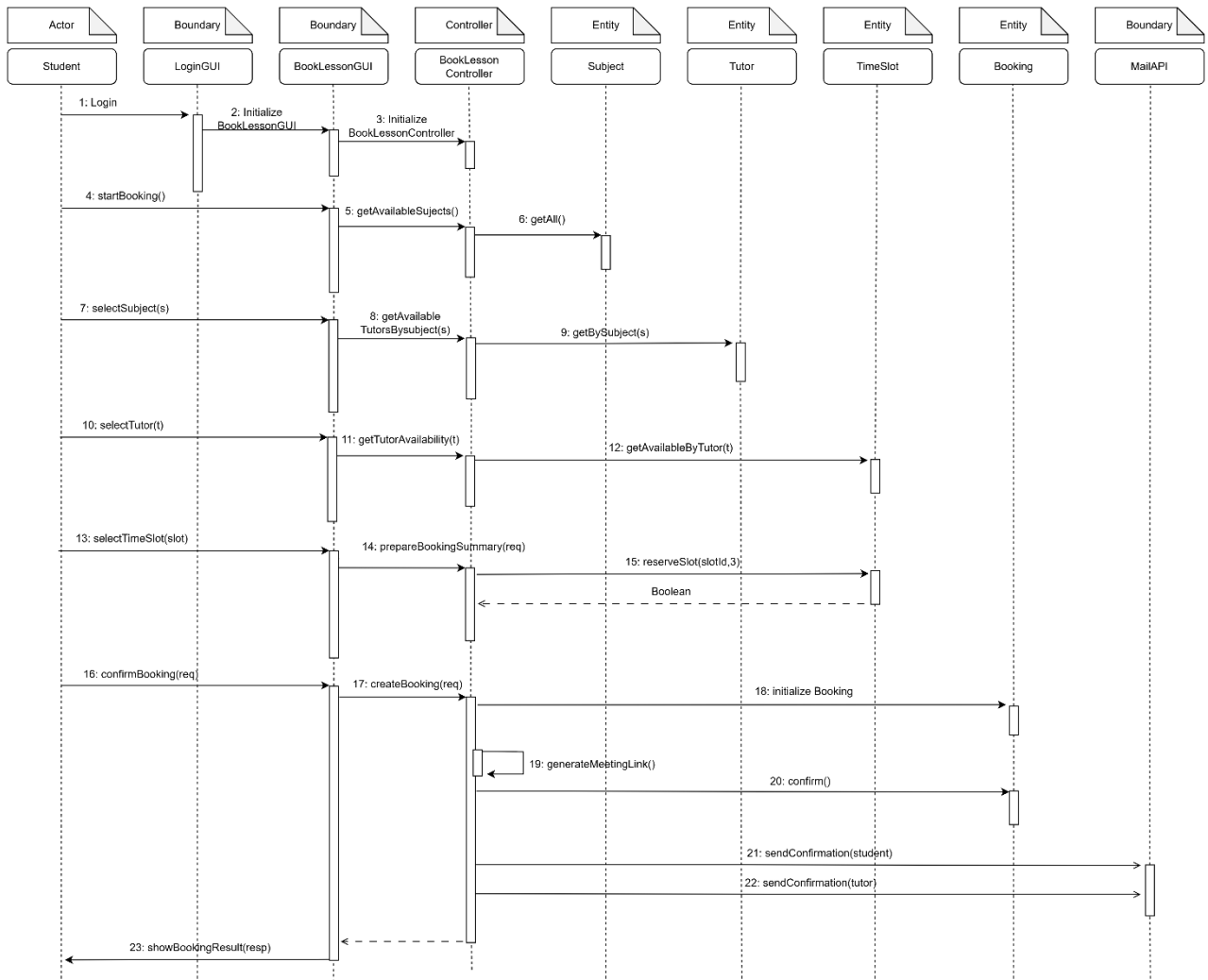
Each screen becomes an independent ConcreteState extending AbstractCLIState, which defines three lifecycle methods: entry() for screen initialization, action() for input handling and transition logic, and exit() as an empty default.

MainCLI acts as the Client, creating a CLIStateMachine and calling start(). CLIStateMachineImpl is the Context, holding the current state and a history stack (Deque<AbstractCLIState>) for forward and backward navigation via push/pop. Each ConcreteState manages its own transitions via a switch-case in action(), keeping each screen self-contained and making the system open to extension without modifying existing classes.

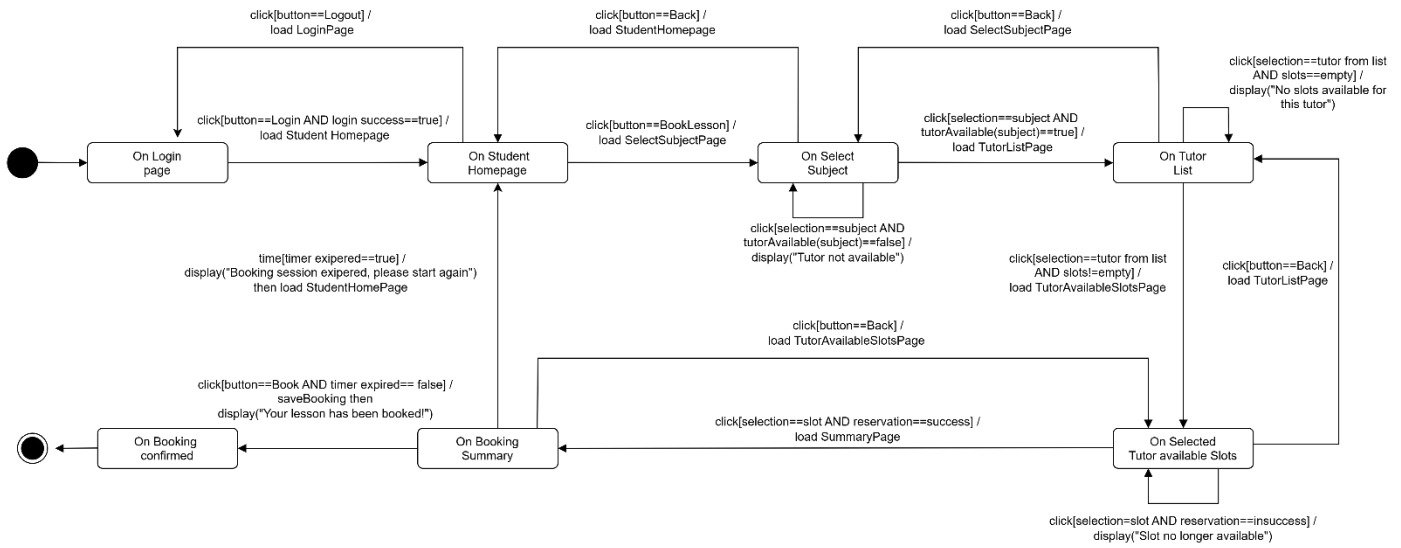
3.3 Activity Diagram



3.4 Sequence Diagram



3.5 State Diagram



4. Testing

1. *Detection of overlapping availability slots*

The slot availability management was tested by verifying that a tutor cannot add two overlapping time slots.

The first slot is added successfully, while a second slot with an overlapping time range correctly raises an `AvailabilityException`.

2. *Booking on an already reserved slot*

The test verifies the 3-minute reservation mechanism: when Student 1 tries to book a lesson, a temporary reservation window is set.

A second attempt by Student 2 on the same slot correctly raises a `BookingException` because the reservation window has not yet expired.

3. *Login with invalid credentials*

The authentication system was tested by attempting a login with an unregistered email address.

This test ensures that the system correctly rejects invalid credentials by raising a `LoginException`, guaranteeing that only registered users can access the platform.

4. *Rejection of duplicate registrations*

The system was verified to prevent two accounts from sharing the same email address.

After a first successful registration, a second attempt with the same email must raise a `RegistrationException`, ensuring the uniqueness of user identities within the platform.

5. Code

GitHub:

<https://github.com/giuliamencaccini/BrainBank>

Sonar Qube Cloud:

https://sonarcloud.io/project/overview?id=giuliamencaccini_BrainBank2

6. Video

<https://youtu.be/BgFq779UlsI?si=DlBx6sgYnTFw2M2A>